Figure 5.11. This is an overview of the architecture for our two-headed actor-critic model. It has two shared linear layers and a branching point where the output of the first two layers is sent to a log-softmax layer of the actor head and also to a ReLU layer of the critic head before finally passing through a tanh layer, which is an activation function that restricts output between –1 and 1. This model returns a 2-tuple of tensors rather than a single tensor. Notice that the critic head is detached (indicated by the dotted line), which means we do not backpropagate from the critic head into the actor head or the beginning of the model. Only the actor backpropagates through the beginning of the model.

Figure 5.12. Within each process, an episode of the game is run using the shared model. The loss is computed within each process, but the optimizer acts to update the shared actor-critic model that is used by each process.

Figure 5.13. The actor and critic have a bit of an adversarial relationship since the actions that the agent take affect the loss of the critic, and the critic makes predictions of state values that get incorporated into the return that affects the training loss of the actor. Hence, the overall loss plot may look chaotic despite the fact that the agent is indeed increasing in performance.

Figure 5.14. The mean episode length over training time for our Monte Carlo distributed advantage actor-critic model. This model is not considered a true critic, since the critic is not bootstrapping during training. As a result, the training performance has high variance.

Figure 5.15. Performance plot for distributed advantage actor-critic with true N-step bootstrapping. Compared to our previous Monte Carlo algorithm, the performance is much smoother due to the more stable critic.

Chapter 6. Alternative optimization methods: Evolutionary algorithms

Figure 6.1. For the past algorithms that we covered, our agent interacted with environment, collected experiences, and then learned from those experiences. We repeated the same process over and over for each epoch until the agent stopped learning.